PriceWise

Web Scraping and Product Price Analysis System

Project explanation, build guide, and external-lab viva preparation

Project overview

PriceWise is a multi-page Python web application that helps shoppers compare prepared product offers across online stores. It loads flexible CSV data, cleans and filters offers, identifies the lowest listed price, presents analytics, gives an explainable deal score, and offers a lightweight machine-learning classification.

The current checked-in version is a data-driven dashboard. Its scraper infrastructure is included, but it is configured in demo mode. During a presentation, state that the app uses the prepared dataset and that the Live Scraping UI is a demonstration simulation.

Dataset snapshot

Metric	Observed value
Offers	1,500
Distinct product names	300
Categories	Beauty, Books, Electronics, Fashion, Home
Platforms	13
Scrape-date span	11 September 2026 to 25 September 2026
Average offer price	Rs. 12,467.27
Price range	Rs. 155.88 to Rs. 163,104.00

How we built the project

1. Data ingestion design

We added BaseScraper for polite collection from permitted sources. It supports robots.txt checks, a declared user agent, request delays, safe redirect handling, local response caching, and retries for temporary failures. It stops on forbidden access and rate limits.

2. Data pipeline

data_pipeline.py completes the expected schema, trims text fields, converts numeric values, validates valid ranges, normalizes dates, removes invalid essential rows, and retains the lowest price for duplicate product and platform records.

3. Flexible data loader

utils/data_loader.py recognizes common CSV column aliases. It cleans currency strings and derives a discount if price and original price exist but discount does not.

4. Analytics engine

utils/analysis.py provides summary statistics, category and platform groupings, discount distribution, platform offer share, product counts, product-level comparison, price history, deal scoring, and prediction functions.

5. Dashboard

The Streamlit pages implement Home, Comparison, Analytics, and About views. Users can search, filter, view product cards, compare platform offers, save a session watchlist, download comparison CSV, and inspect interactive Plotly charts.

6. Deal recommendation and ML

The heuristic deal score uses lower price, discount, rating, and availability. Logistic Regression classifies a good deal as an offer whose price is below the loaded dataset median. Categorical features use one-hot encoding.

7. Testing and packaging

Unit tests cover invalid prices, discounts, ratings, duplicate handling, 429 stopping behavior, and retry behavior for temporary server errors. A Dockerfile is included for container deployment.

Demonstration sequence

1. Install dependencies from requirements.txt and start the Streamlit application through app.py.

2. Open Home, show category, platform, and price filters, then search for a product.

3. Open View Details and explain the price table, lowest price, highest price, average, potential saving, chart, and CSV export.

4. Show the category comparison option when an exact product is unavailable on several platforms.

5. Open Analytics and explain the category, platform, discount, offer-share, and product-count charts.

6. Demonstrate the Deal Quality form, while explaining that its target is below-median price in this dataset.

External lab viva questions

Use these as answer cues. Explain in your own words and do not claim live scraping or fixed ML metrics that have not been reproduced from the current code.

1. What problem does PriceWise solve?

Answer cue: It reduces the effort of checking the same product across several online stores and highlights price differences.

2. What is the project objective?

Answer cue: To collect or load product offers, clean them, compare prices, show analytics, and recommend attractive deals.

3. Who are the users?

Answer cue: Online shoppers who want a quick, data-driven price comparison before buying.

4. What are the four architectural layers?

Answer cue: Ingestion, data pipeline, analytics and ML, and the Streamlit presentation layer.

5. Which language and framework are used?

Answer cue: Python is the language and Streamlit builds the interactive web interface.

6. Why Streamlit?

Answer cue: It lets us build a multi-page data application with Python, widgets, charts, and fast reruns.

7. What is the role of Pandas?

Answer cue: Pandas loads CSV data, cleans fields, filters offers, groups records, and calculates summaries.

8. What is Plotly used for?

Answer cue: It creates interactive bar, line, pie, and distribution charts in the dashboard.

9. What is the primary data source today?

Answer cue: The checked-in data/pricewise_data.csv file. It contains prepared product offers used by the dashboard.

10. Does the current project scrape live retail websites?

Answer cue: No. scraper_config.py is set to demo mode. The current UI uses the prepared CSV; the Live Scraping button is also a simulation.

11. Why is the scraper separated from the UI?

Answer cue: It decouples slower data collection from fast dashboard interaction and makes each part easier to maintain.

12. What fields are stored for an offer?

Answer cue: Product name, category, platform, price, original price, discount, rating, availability, URL, scrape date, and optional image path.

13. How is schema flexibility achieved?

Answer cue: The loader maps common column aliases such as name/title and price/current_price to canonical fields.

14. What happens if optional fields are absent?

Answer cue: The application hides dependent UI or analysis safely instead of failing.

15. How are numeric values cleaned?

Answer cue: Currency symbols, commas, and whitespace are removed, then values are converted with invalid values becoming missing.

16. How does the pipeline validate data?

Answer cue: It limits valid price, discount, and rating ranges, normalizes dates, and removes rows without product name or price.

17. How are duplicates handled?

Answer cue: Exact duplicates are removed; duplicate product name and platform records keep the lowest price.

18. Why keep the lowest duplicate price?

Answer cue: For the same listed product on the same platform, the lowest usable observed offer is the most useful comparison value.

19. What is the BaseScraper?

Answer cue: A reusable HTTP component that handles robots.txt checks, delays, caching, redirects, retries, and errors.

20. How does it behave on HTTP 429?

Answer cue: It stops that source immediately and reports rate limiting rather than repeatedly requesting it.

21. How are temporary server failures handled?

Answer cue: 5xx errors and connection or timeout failures can retry with exponential backoff, up to the configured retry count.

22. How does it respect website policies?

Answer cue: It checks robots.txt, uses a declared user agent, applies a request delay, and has no CAPTCHA bypass logic.

23. What does caching do?

Answer cue: It saves successful responses locally so identical requests can be reused and fewer requests are sent.

24. How does search work?

Answer cue: The home page builds one best-price record per product, then filters it with typed search and fuzzy suggestions.

25. Which filters are available?

Answer cue: Category, platform, and price range; they narrow the dataset before listings and analytics are calculated.

26. How does exact product comparison work?

Answer cue: It matches the selected product name, groups records by platform, and uses the lowest platform price.

27. How is potential saving calculated?

Answer cue: Potential saving equals the average price across available platforms minus the lowest price.

28. What happens when exact products cannot be matched?

Answer cue: The category comparison tab shows the cheapest product on each platform within the selected category.

29. What analytics are available?

Answer cue: Price summary, category and platform averages, discount distribution, product counts, and platform offer share.

30. What does platform market share mean here?

Answer cue: It is the share of offers in this dataset, not the company real commercial market share.

31. How does price history work?

Answer cue: When scrape dates exist, the application groups a selected product by date using the day's minimum observed price.

32. What is the watchlist?

Answer cue: A session-level list of product names saved by a user during the current Streamlit session.

33. What is the deal score?

Answer cue: A weighted heuristic combining lower price 40%, discount 25%, rating 20%, and availability 15%.

34. Why normalize score components?

Answer cue: Price, percentage discount, and ratings use different scales; normalization puts them onto a comparable 0-1 scale.

35. What is MCDA in this project?

Answer cue: Multi-Criteria Decision Analysis: it combines several shopping criteria into one deal score.

36. What ML algorithm is used?

Answer cue: Logistic Regression classifies whether an offer is a good deal according to the project target definition.

37. What are the ML input features?

Answer cue: Category, platform, discount, and rating, when those columns are available.

38. What target label does the code use?

Answer cue: An offer is labeled good when its price is below the median price of the loaded dataset.

39. How are categorical features prepared?

Answer cue: OneHotEncoder converts category and platform values into binary indicator columns and ignores unseen values safely.

40. How is the model evaluated in the app?

Answer cue: The data is split 80:20 with random_state 42 and the UI reports held-out accuracy.

41. Is good-deal prediction the same as best-deal score?

Answer cue: No. The score is a transparent weighted rule; the classifier learns a below-median label from the dataset.

42. What are the ML limitations?

Answer cue: The target is relative to the dataset, not a verified real bargain; accuracy can change with data quality and class balance.

43. Why use handle_unknown ignore?

Answer cue: It prevents failure if a user selects a category or platform not seen while the model was trained.

44. What dataset scale is used?

Answer cue: 1,500 offers representing 300 distinct product names across five categories and 13 platforms.

45. Which categories are covered?

Answer cue: Beauty, Books, Electronics, Fashion, and Home.

46. What tests are included?

Answer cue: Pipeline tests cover invalid values and duplicate rules; scraper tests cover 429 handling and 5xx retry behavior.

47. How can it be deployed?

Answer cue: With Streamlit Community Cloud or the supplied Dockerfile, which starts Streamlit on port 8501.

48. What are major limitations?

Answer cue: No live sources are configured, matching uses exact names, prices can vary by location or payment offer, and the watchlist is not persistent.

49. What would you improve first?

Answer cue: Use authorized APIs or permitted sources, add canonical product matching, persist history in a database, and add price-drop alerts.

50. How do you defend the project value?

Answer cue: It demonstrates the full flow from responsible ingestion design and data cleaning to consumer-facing analytics and explainable recommendations.